

Tic Tac Toe Use Case Descriptions

Use Case 1: Make a Move

Iteration: 1

Primary Actor: Player

Goal: Allows the player to make a move and place X or O on the board.

Preconditions:

1. It's the player's turn.
2. The game is in progress
3. There are empty cells on the board.

Trigger: A cell is selected by the player.

Scenario:

1. The player will select a cell to place X or O.
2. The system will check if that cell is available.
3. If it is available, the system will place the X or O.
4. The system will check for game over.
5. If the game is not over the system will change the active player.

Postconditions:

1. The board updates with the player's move.
2. The system will change the active player to the next player.

Exceptions: The system will notify the player if it is an invalid move (cell is not empty).

Priority: High

Availability: Once game begins

Frequency of use: Every turn

Channel to actors: Through the game interface

Secondary actors: System

Channel to Secondary actors: Game logic

Open issues: Should there be an undo option/feature for the moves?

Use Case 2: Check for game over

Iteration: 1

Primary Actor: System

Goal: Checks if the game should be declared over or not.

Preconditions: A move has been made.

Trigger: System check after a player has made a move.

Scenario:

1. The system will check for win conditions.
2. If a player has won, the system will prompt that and end the game.
3. If there is no winner, the system will check for a draw.
4. If the game is a draw the system will declare that and end the game.
5. If none of these conditions meet, the game continues.

Postconditions: If the game is won or is a draw then the game ends, or else it continues.

Exceptions: If the system is unable to check for a win or a draw correctly, the game will continue wrongly.

Priority: High

Availability: After every move.

Frequency of use: After every move.

Channel to actors: Game logic.

Secondary actors: None

Channel to Secondary actors: N/A

Open issues: None

Use Case 3: Check for win

Iteration: 1

Primary Actor: System

Goal: Checks if a player has won the game.

Preconditions: A move has been made.

Trigger: System check after a player has made a move.

Scenario:

1. The system will check for any rows or columns or any diagonals with three matching marks (X's or O's).
2. If a player has won, the system will prompt a win.

Postconditions: If the game is won then the game ends and the player is declared a winner.

Exceptions: If the system is unable to check for a win or declares an incorrect win.

Priority: High

Availability: After every move.

Frequency of use: Once after every move. Multiple times in a game.

Channel to actors: Game logic.

Secondary actors: None

Channel to Secondary actors: N/A

Open issues: None

Use Case 4: Check for draw

Iteration: 1

Primary Actor: System

Goal: Checks if the game should be declared a draw or not.

Preconditions:

1. A move has been made.
2. No player has won.

Trigger: System detects a full board.

Scenario:

1. The system detects a full board.
2. If no empty cell remains, and no win condition is met the game declares a draw.
3. The game ends with the system prompting a draw message.

Postconditions: If a draw occurs then the game ends.

Exceptions: If the system incorrectly declares a draw before all cells are full.

Priority: High

Availability: After every move.

Frequency of use: Only when the board is full.

Channel to actors: Game logic.

Secondary actors: None

Channel to Secondary actors: N/A

Open issues: None

Use Case 5: Change active player

Iteration: 1

Primary Actor: System

Goal: Allows the system to change the turn between players.

Preconditions:

1. Player has successfully made a move.
2. The game is not over.

Trigger: System calls after a valid move.

Scenario:

1. The system identifies the current player.
2. The system switches turn to the next player.
3. The system prompts the new player to make a move.

Postconditions: The player has changed and the new player can make a move.

Exceptions: If the system fails to make the switch, then the same player will play twice.

Priority: High

Availability: After every valid move.

Frequency of use: Once every move.

Channel to actors: Game logic.

Secondary actors: Player

Channel to Secondary actors: Game interface.

Open issues: Should there be a visual indicator showing which player's turn?

Use Case 6: Forfeit

Iteration: 1

Primary Actor: Player

Goal: Allows the player to surrender and end the game early.

Preconditions: The game is in progress.

Trigger: The player chooses the option to forfeit.

Scenario:

1. The player selects the forfeit option.
2. The system asks for confirmation for action.
3. The game ends and the opponent wins.

Postconditions: The game ends and the opponent wins.

Exceptions: If the system is unable to detect the forfeit, and continues the game.

Priority: Medium

Availability: Anytime during the game.

Frequency of use: Only when a player wants to quit.

Channel to actors: Game menu interface.

Secondary actors: None

Channel to Secondary actors: N/A

Open issues: None